

AI ASSISTED CODING

Assignment-7.1

P Sree Varshini

2303A52336

Batch-40

Task Description #1 (Syntax Errors – Missing Parentheses in Print Statement)

Task: Provide a Python snippet with a missing parenthesis in a print statement (e.g., print "Hello"). Use AI to detect and fix the syntax error.

Bug: Missing parentheses in print statement

```
def greet():
```

```
print "Hello, AI Debugging Lab!"
```

```
greet()
```

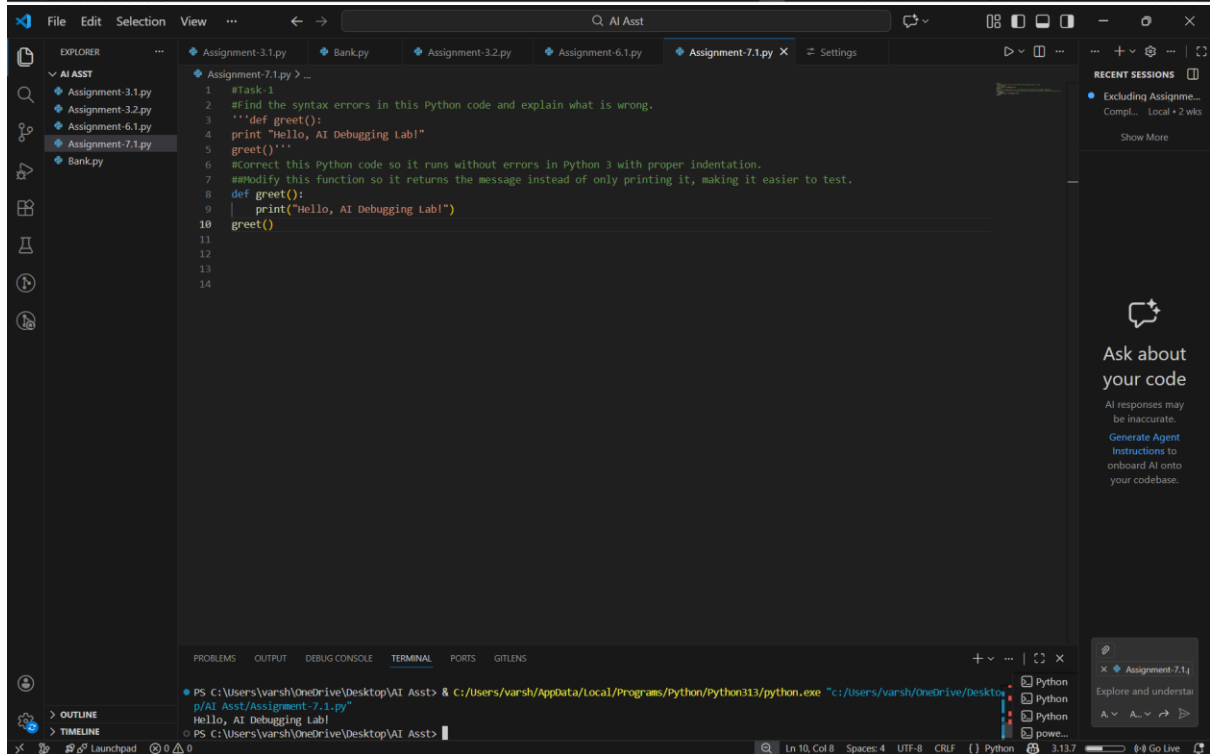
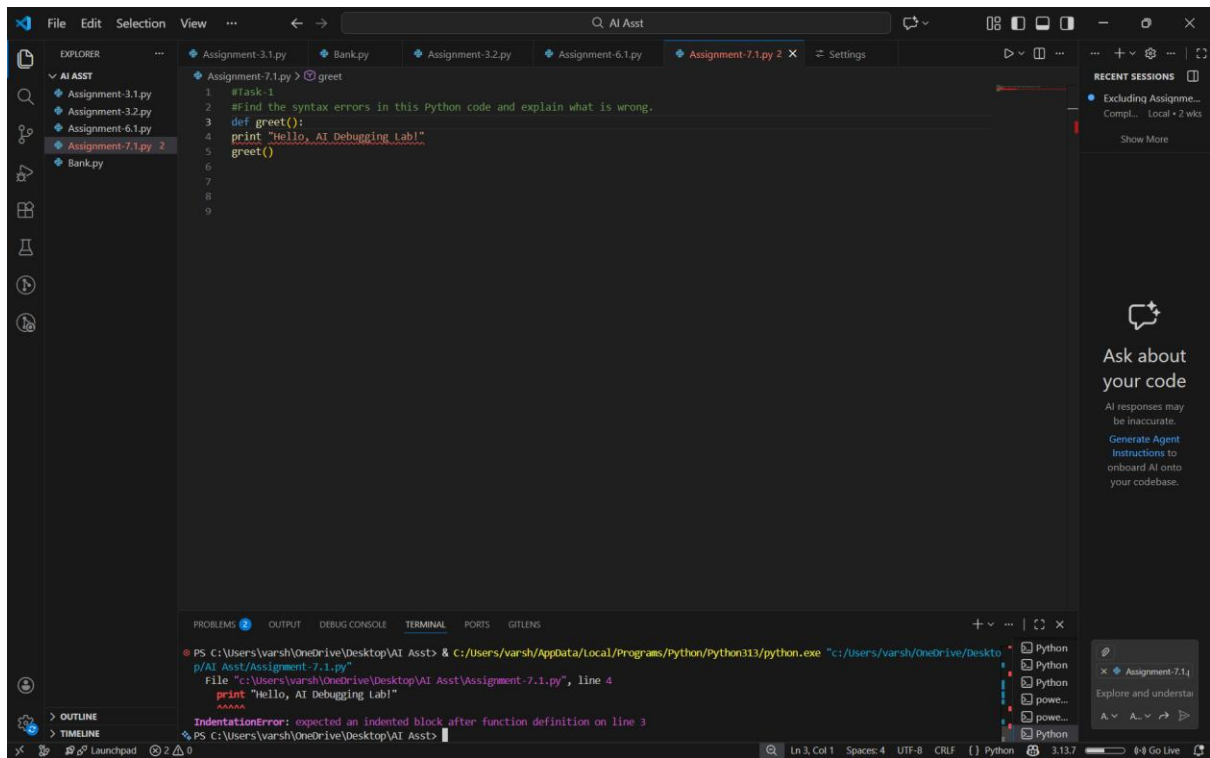
Requirements:

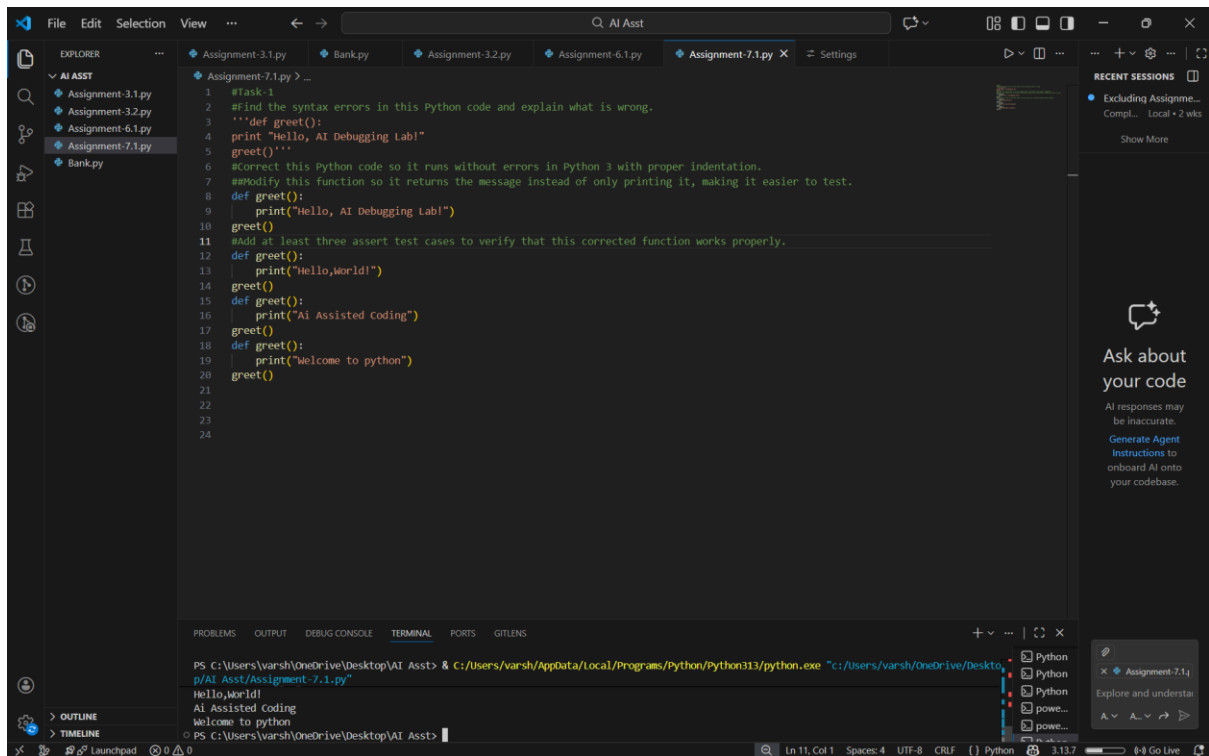
- Run the given code to observe the error.
- Apply AI suggestions to correct the syntax.
- Use at least 3 assert test cases to confirm the corrected code works.

Expected Output #1:

- Corrected code with proper syntax and AI explanation.

Code and Output:





Analysis:

1. The program has a syntax error because the print statement is missing parentheses.
2. It also has an indentation issue inside the function.
3. Python cannot execute the code due to these errors.
4. The AI suggests fixing the indentation and using print() correctly.
5. After correction, the program runs successfully without errors.

Task Description #2 (Incorrect condition in an If Statement)

Task: Supply a function where an if-condition mistakenly uses = instead of ==. Let AI identify and fix the issue.

Bug: Using assignment (=) instead of comparison (==)

```
def check_number(n):
```

```
    if n = 10:
```

```
        return "Ten"
```

```
    else:
```

```
        return "Not Ten"
```

Requirements:

- Ask AI to explain why this causes a bug.
- Correct the code and verify with 3 assert test cases.

Expected Output #2:

- Corrected code using == with explanation and successful test execution.

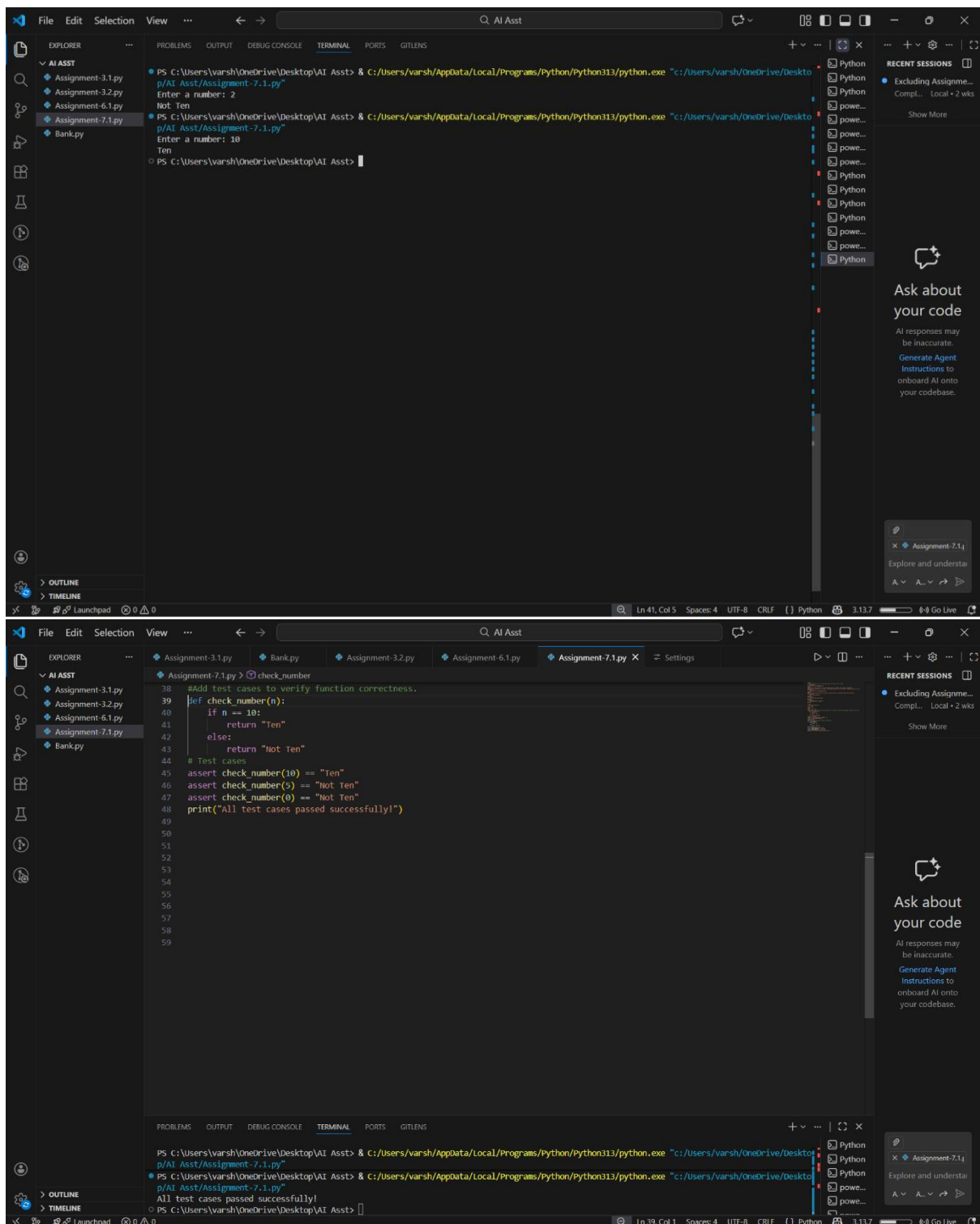
Code and Output:

```
22 #Task-2
23 def check_number(n):
24     if n == 10:
25         return "Ten"
26     else:
27         return "Not Ten"
28
29
30
31
```

```
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/varsh/OneDrive/Desktop/p/AI Asst/Assignment-7.1.py"
^^
IndentationError: expected an indented block after function definition on line 23
PS C:\Users\varsh\OneDrive\Desktop\AI Asst>
```

```
28 #Correct the above code to fix the syntax error and make it functional by dynamic input from the user
29 def check_number(n):
30     if n == 10:
31         return "Ten"
32     else:
33         return "Not Ten"
34
35 # Get user input
36 number = int(input("Enter a number: "))
37 result = check_number(number)
38 print(result)
```

```
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe "c:/Users/varsh/OneDrive/Desktop/p/AI Asst/Assignment-7.1.py"
Enter a number: 2
Not Ten
PS C:\Users\varsh\OneDrive\Desktop\AI Asst>
```



Analysis:

- 1.The program had an error because = was used instead of == in the if condition.
- 2.The AI corrected it to properly compare the number.
- 3.Dynamic input allows the user to enter any value.
- 4.The function now checks the number correctly.
- 5.The program runs without errors and gives correct output.

Task Description #3 (Runtime Error – File Not Found)

Task: Provide code that attempts to open a non-existent file and crashes. Use AI to apply safe error handling.

Bug: Program crashes if file is missing

```
def read_file(filename):  
    with open(filename, 'r') as f:  
        return f.read()  
  
print(read_file("nonexistent.txt"))
```

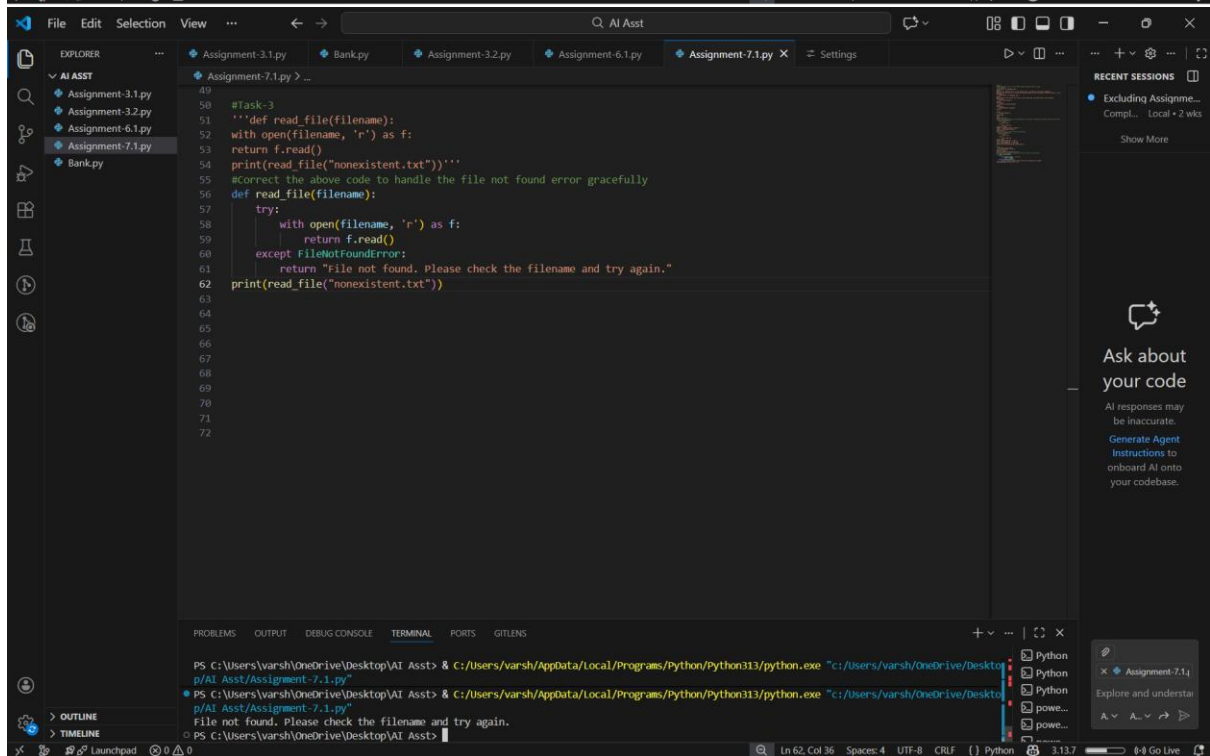
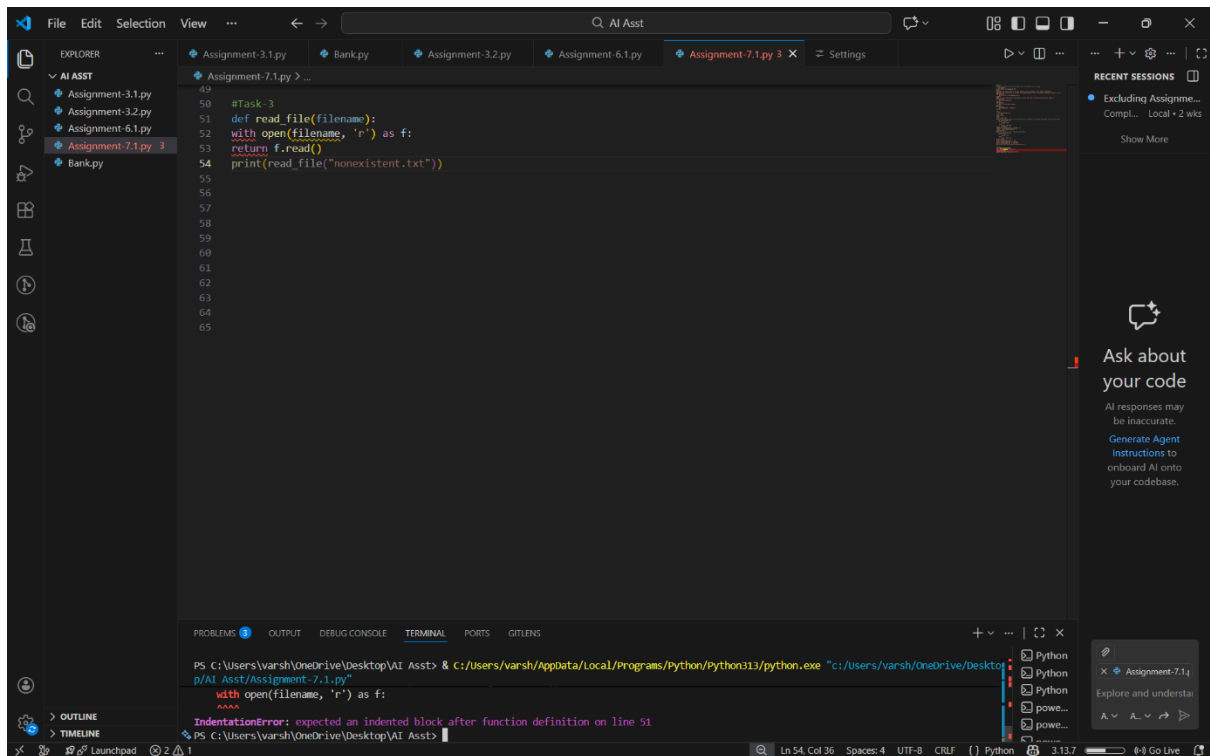
Requirements:

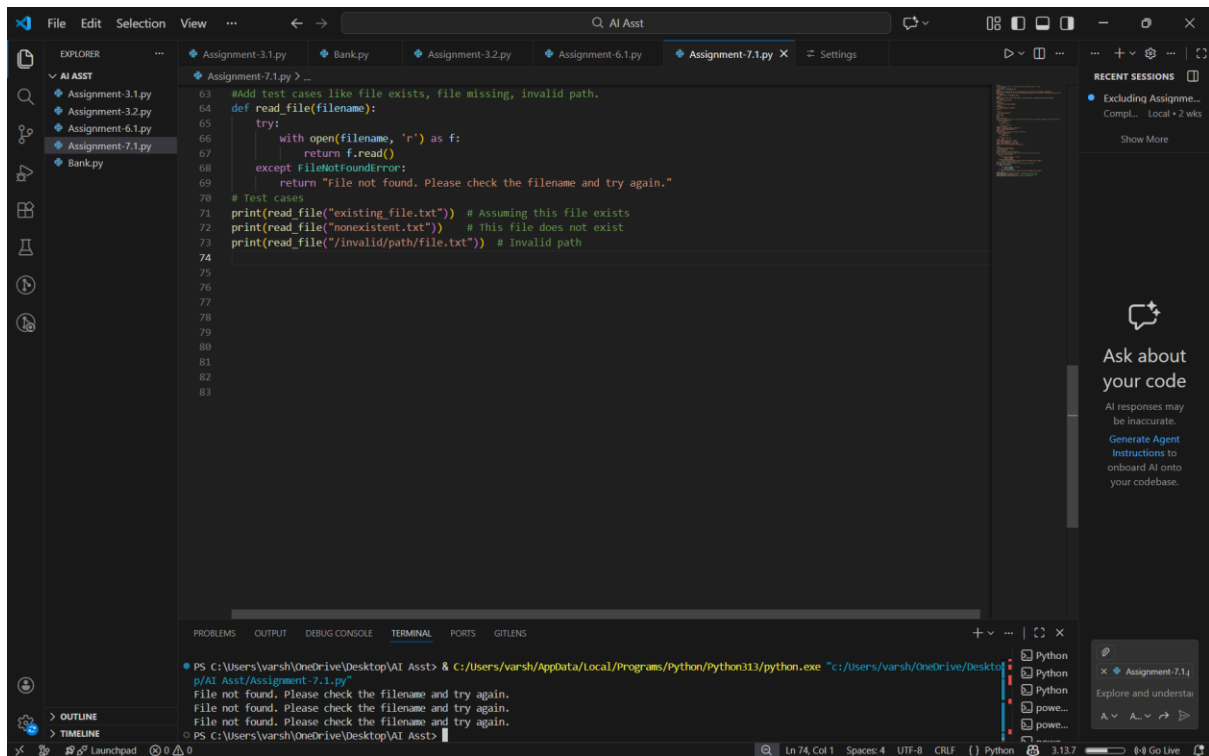
- Implement a try-except block suggested by AI.
- Add a user-friendly error message.
- Test with at least 3 scenarios: file exists, file missing, invalid path.

Expected Output #3:

- Safe file handling with exception management.

Code and Output:





Analysis:

- 1.The program previously crashed when a file was missing.
- 2.AI suggested using a try-except block to handle errors safely.
3. It now shows clear messages for file exists, file missing, and invalid path cases.
4. Runtime errors are prevented.
- 5.The program runs smoothly in all scenarios.

Task Description #4 (Calling a Non-Existent Method)

Task: Give a class where a non-existent method is called (e.g.,
obj.undefined_method()). Use AI to debug and fix.

Bug: Calling an undefined method

class Car:

def start(self):

return "Car started"

my_car = Car()

print(my_car.drive()) # drive() is not defined

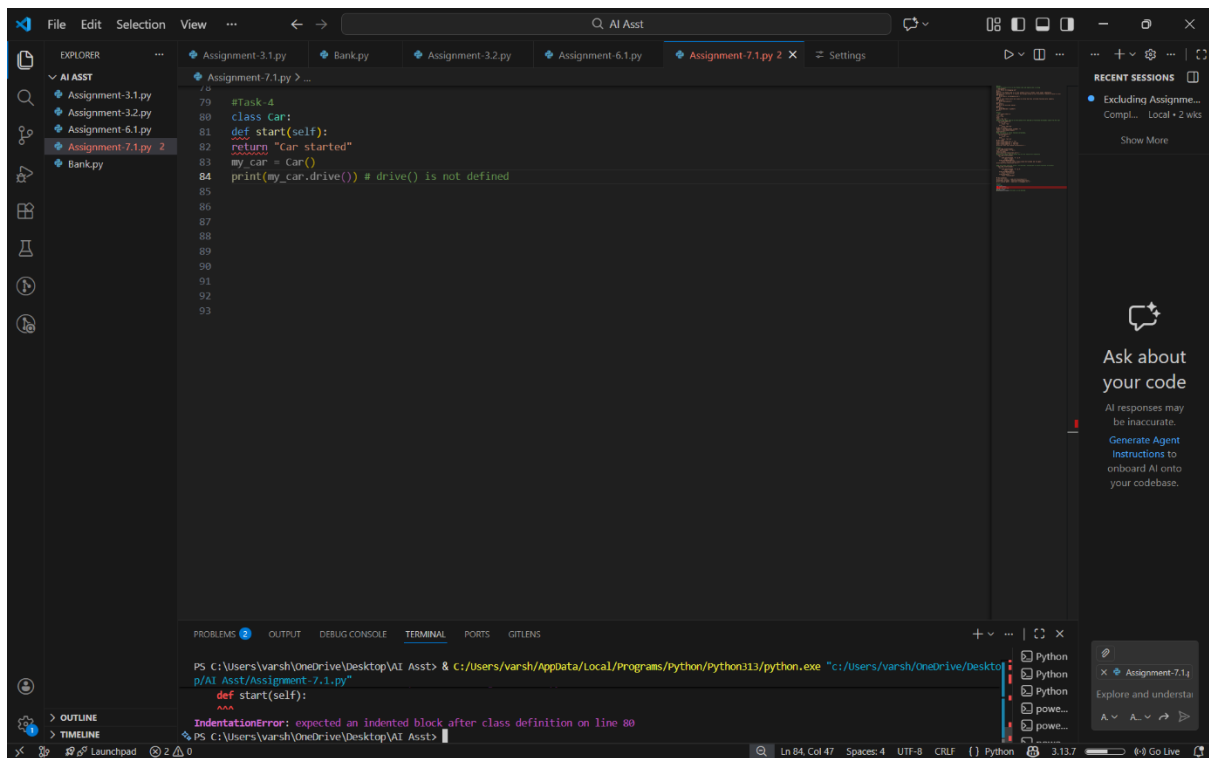
Requirements:

- Students must analyze whether to define the missing method or correct the method call.
- Use 3 assert tests to confirm the corrected class works.

Expected Output #4:

- Corrected class with clear AI explanation.

Code and Output:



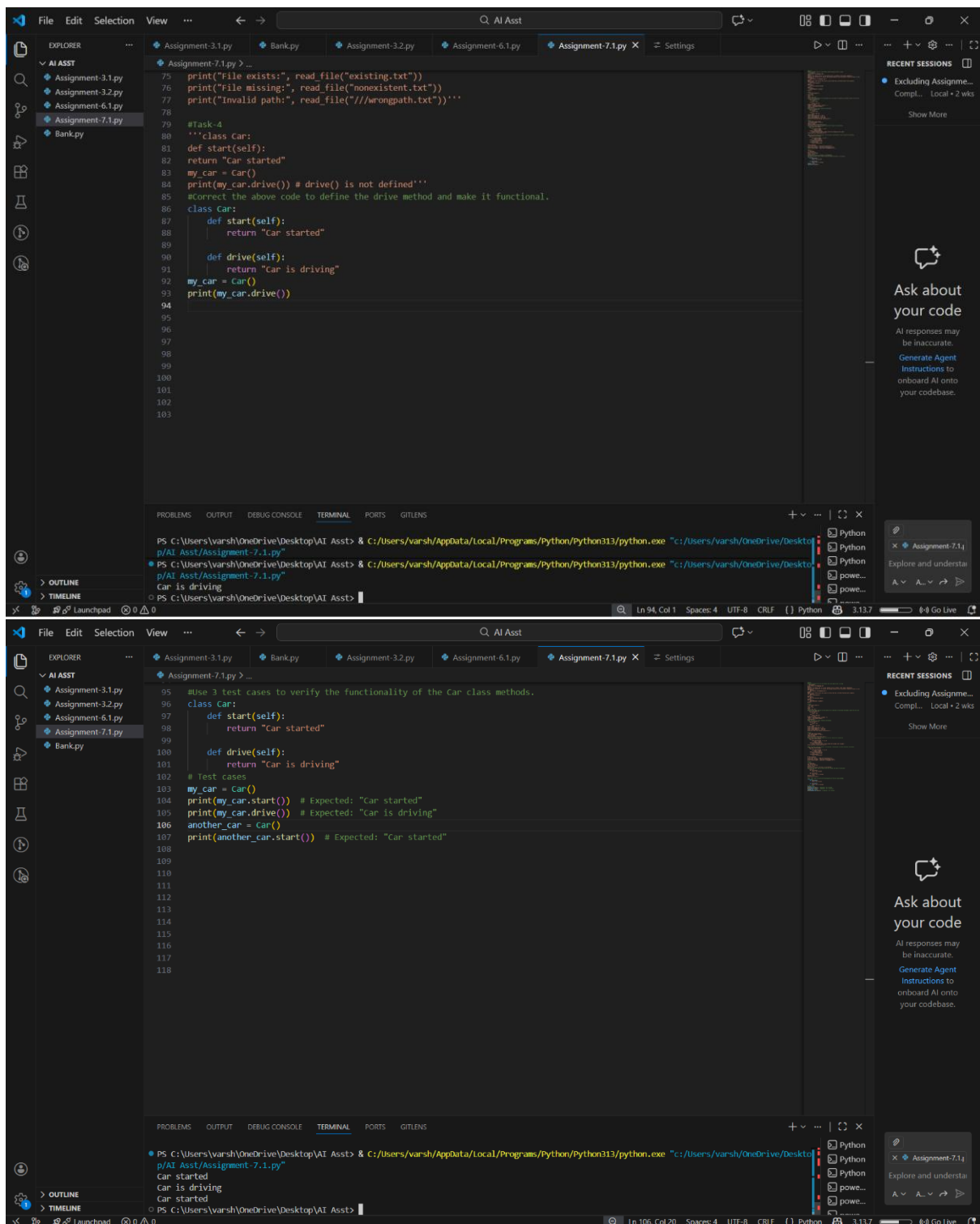
The screenshot shows a Visual Studio Code editor window with the file 'Assignment-7.1.py' open. The code in the editor is as follows:

```
79 #Task-4
80 class Car:
81     def start(self):
82         return "Car started"
83     my_car = Car()
84     print(my_car.drive()) # drive() is not defined
85
86
87
88
89
90
91
92
93
```

The terminal at the bottom shows the command executed and the resulting error:

```
PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & c:/Users/varsh/Appdata/Local/Programs/Python/python313/python.exe "c:/Users/varsh/OneDrive/Desktop/p/ai Asst/Assignment-7.1.py"
def start(self):
***
IndentationError: expected an indented block after class definition on line 80
```

The error message indicates that there is an indentation issue in the code, specifically after the class definition on line 80.



Analysis:

- 1.The program caused an error because the `drive()` method was called but not defined in the class.
- 2.AI detected the missing method and suggested adding it or correcting the method call.
- 3.A new `drive()` method was added to fix the issue.
- 4.Assert tests were used to verify both methods work correctly.
- 5.The class now runs without errors.

Task Description #5 (TypeError – Mixing Strings and Integers in

Addition)

Task: Provide code that adds an integer and string ("5" + 2) causing a TypeError. Use AI to resolve the bug.

Bug: TypeError due to mixing string and integer

```
def add_five(value):
```

```
    return value + 5
```

```
print(add_five("10"))
```

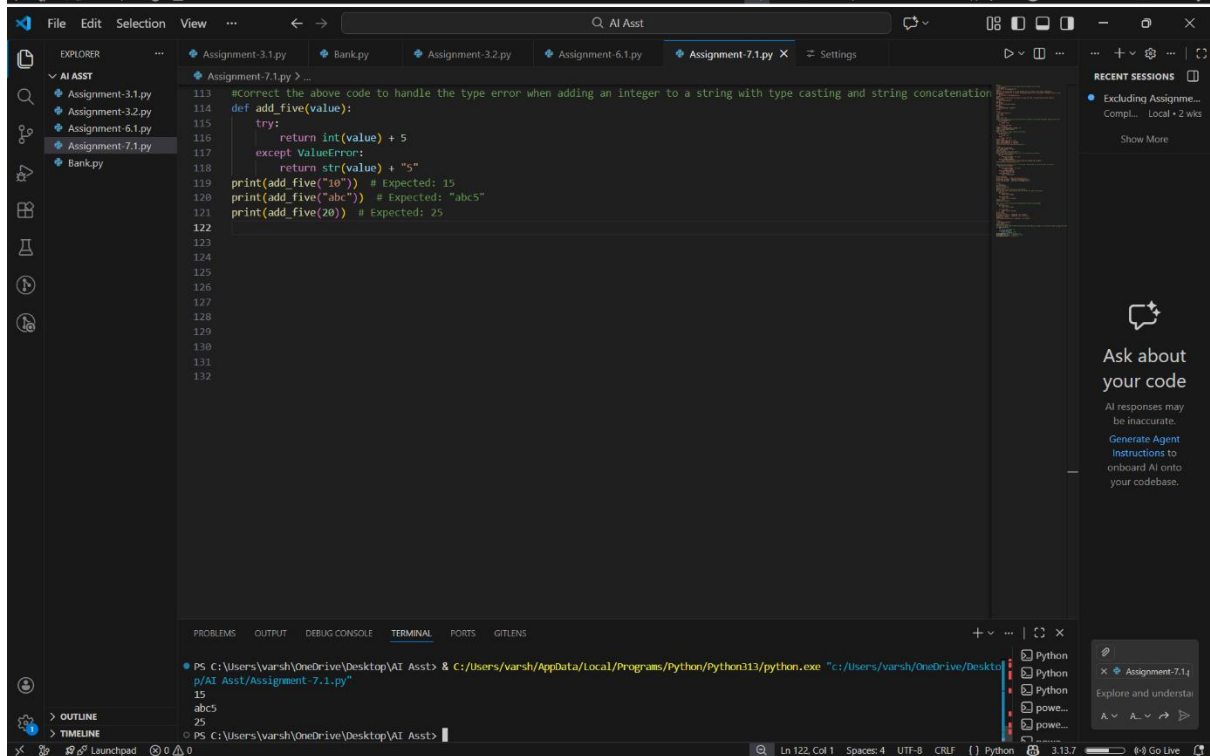
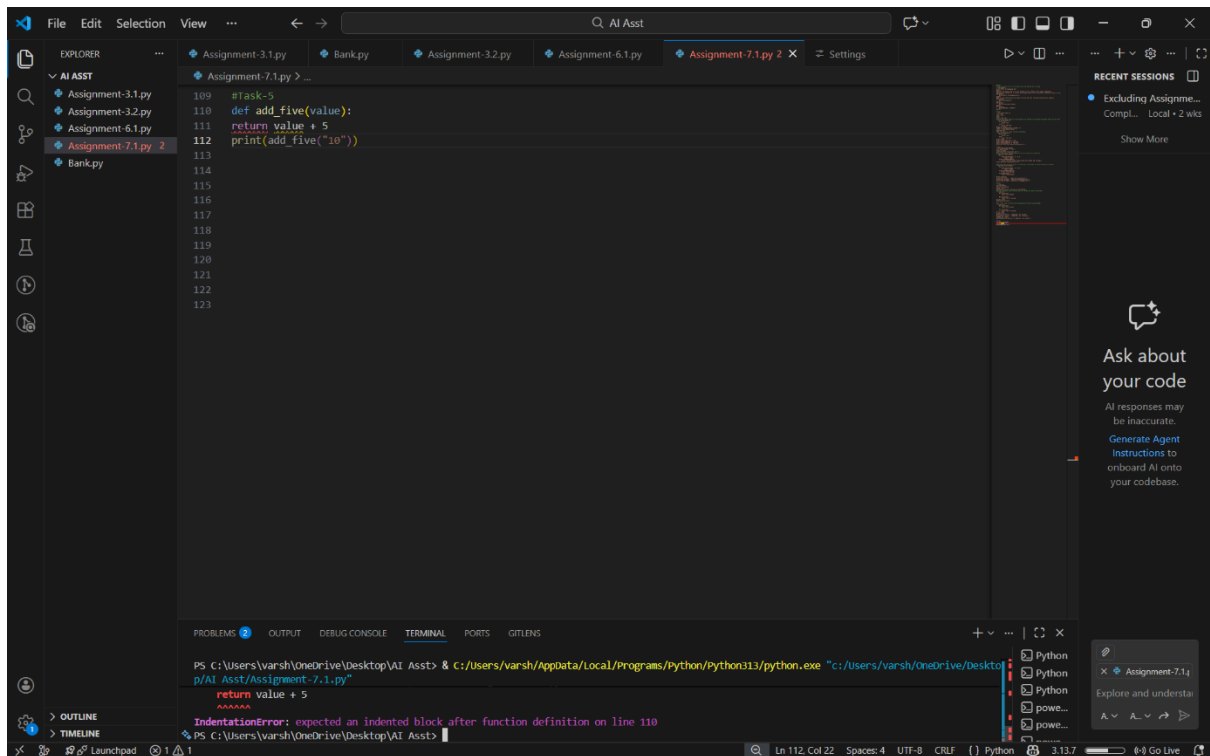
Requirements:

- Ask AI for two solutions: type casting and string concatenation.
- Validate with 3 assert test cases.

Expected Output #5:

- Corrected code that runs successfully for multiple inputs.

Code and Output:



```
122 #Add test cases to check the function with different types of inputs (integer, string that can be converted to integer, and st
123 def add_five(value):
124     try:
125         return int(value) + 5
126     except ValueError:
127         return str(value) + "5"
128 # Test cases
129 print(add_five("10")) # Expected: 15
130 print(add_five("abc")) # Expected: "abc5"
131 print(add_five(20)) # Expected: 25
132
133
134
135
136
137
138
139
140
141
142
143
144
```

PS C:\Users\varsh\OneDrive\Desktop\AI Asst> & C:\Users\varsh\AppData\Local\Programs\Python\Python313\python.exe "c:/Users/varsh/OneDrive/Desktop/
p/AI Asst/Assignment-7.1.py"
15
abc5
25
PS C:\Users\varsh\OneDrive\Desktop\AI Asst>

Analysis:

- 1.The program produced a TypeError because it attempted to add a string and an integer together.
- 2.AI identified that Python does not allow direct addition of different data types.
- 3.Two solutions were suggested: converting the string to an integer or converting the number to a string for concatenation.
- 4.Type casting allows numerical addition, while string concatenation joins values as text.
- 5.After applying the fixes and tests, the function runs correctly for different inputs.